Explainable Cervical Cell Classification: A Preprocessing-Enhanced Deep Learning Approach with Grad-CAM using SIPaKMeD - Progress Report #1

Ayşe Ece Akkurt
Electrical-Electronics Engineering
Abdullah Gül University
Kayseri, Türkiye
ayseece.akkurt@agu.edu.tr

Selin Nisa Akgöl
Electrical-Electronics Engineering
Abdullah Gül University
Kayseri, Türkiye
selinnisa.akgol@agu.edu.tr

Muhammet Talha Akşit
Electrical-Electronics Engineering
Abdullah Gül University
Kayseri, Türkiye
muhammettalha.aksit@agu.edu.tr

I. Dataset

The first part of the project requires us to organize the dataset into an easily understandable and usable order. The dataset is downloaded from https://www.cse.uoi.gr/ marina/sipakmed.html and uploaded to Google Drive and mounted to the file tree. The data is in zipped form (im_cellname.7z) and is extracted into a newly formed folder 'SIPaKMeD_extracted'. This folder includes 5 classes in the organization that the zipped files have, and includes the manually drawn borders of the nucleus and cytoplasm by specialists of each and every cell. The images are in .bmp format, and the hand-drawn borders that we aim to use for segmentation comparison are in .dat format, containing the pixel coordinates.

Right after extraction, an example cell cluster image is displayed to confirm the images can be called from the dataset.

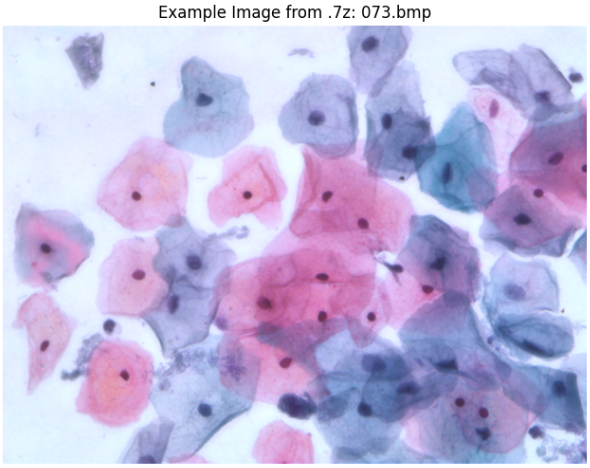

Fig. 1: Example cluster image from the dataset

Later, the images are resized into 224x224 to be prepared to match the required entrance size of the CNN architecture. After this point, the image format is changed to .png to be used more comfortably in the preprocessing stage. PNG files are compressed without compromising the quality of the image, but save space.

The dataset is split into train/validation/test groups and saved in another folder 'SIPaKMeD_split_224' in 70/15/15, while still preserving 5 classes in folders corresponding to their names. The dataset is now ready to be preprocessed and to train the model, as well as to validate and test it afterward. However, this week will not cover the training and transfer learning parts. This week's goal is to see whether preprocessing is a necessity and test the success of the automated segmentation. After that is confirmed or not, we will proceed with a smaller dataset to evaluate the performance of our model.

II. Preprocessing

A. Median Filtering and CLAHE

After the dimensions of the cropped cell images were set to 224×224, median filtering was applied as the first preprocessing step. The purpose of this step was to reduce potential noise in the microscopic images while preserving important cellular structures such as the nuclear boundary and cytoplasmic edges. Median filtering replaces a pixel's value with the median value of its local neighborhood, rather than with the average of its neighbors. This is useful for removing small amounts of noise without causing significant blurring.

Figure 2 shows resized sample images taken from the training set, while Figure 3 shows the same samples after median filtering has been applied. To better assess the filter's effect, the comparison was made using the same image selected from each cell class.

Upon examining the images, it was observed that the median filter did not produce a noticeable difference. After applying the median filter, the overall appearance of the cells, nuclei, and cytoplasmic regions remained nearly the same. In some cell images, the filter slightly smoothed out certain tissues; however, this improvement does not imply that median-filtered images should be used in subsequent stages.

```python
for i, class_name in enumerate(classes):
    class_path = os.path.join(split_base, "train",
        class_name)

    if class_name in sample_images_for_comparison:
        sample_image = sample_images_for_comparison[
            class_name]
        image_path = os.path.join(class_path,
            sample_image)
        img = Image.open(image_path)
```

Listing 1: Sample image selection for comparison

This part of the code was used to select the same sample image from each cell class. This was important because the aim was to compare the effect of median filtering on the same images. If different images were used before and after filtering, the comparison would not be reliable.

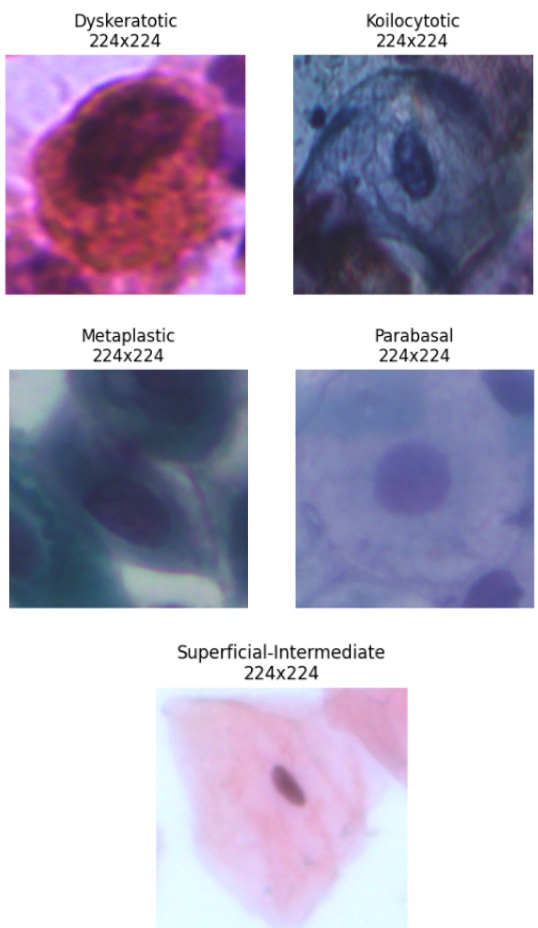

Fig. 2: Sample resized images from the training set before applying median filtering.

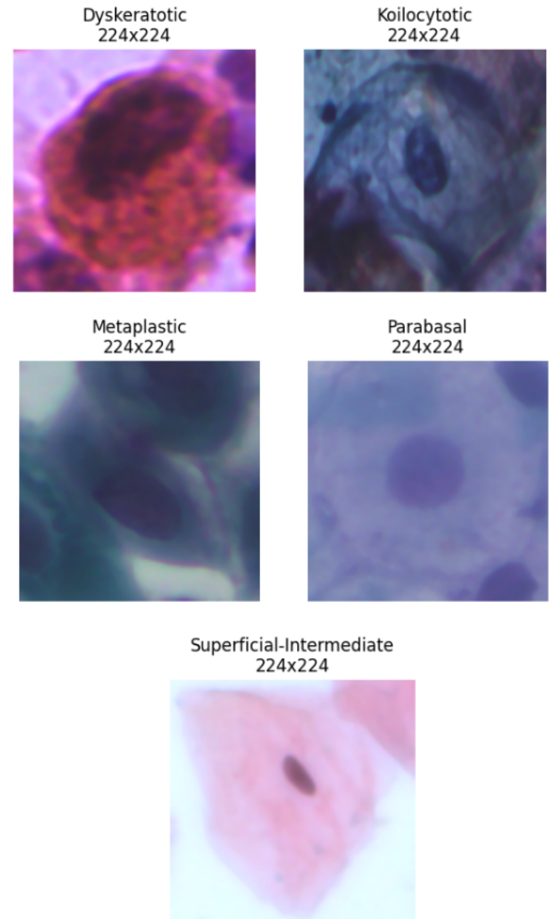

Fig. 3: Sample resized images from the training set after applying median filtering

After trying median filtering, CLAHE was also applied as a preprocessing method. CLAHE stands for Contrast-Limited Adaptive Histogram Equalization. The primary reason for using CLAHE is to improve the contrast of cell images. In microscopic images, some areas may appear too bright, too dark, or have low contrast. This can make it difficult to distinguish important structures such as the nucleus and cytoplasm.

Unlike standard histogram equalization, CLAHE does not apply contrast enhancement to the entire image at once. Instead, it divides the image into smaller local regions and enhances the contrast in each region separately.

CLAHE was tested in two different ways. First, CLAHE was applied directly to the resized original images. Second, CLAHE was applied after median filtering. The aim was to check whether applying median filtering before CLAHE would improve the final image quality. Figure 4 shows the images after applying CLAHE to the original resized images. Figure 5 shows the images after applying CLAHE to the median filtered images.

When the results were compared, it became clear once again that applying a median filter before the CLAHE process did not provide any significant additional improvement.

As a result, CLAHE made the cell structures more visible by increasing the local contrast of the images. Especially in some samples, the nucleus and cytoplasm boundaries became easier to observe compared to the original resized images. This made CLAHE a useful preprocessing step before the segmentation stage.

```python
clahe_on_filtered_image_path = os.path.join(
    clahe_on_filtered_output_base,
    "train",
    class_name,
    sample_file_name
)

clahe_on_filtered_img = Image.open(
    clahe_on_filtered_image_path
).convert("RGB")
```

Listing 2: Loading CLAHE processed image after median filtering

This part of the code loads the image where CLAHE was applied after median filtering. This was used to check whether using median filtering before CLAHE improved the image more than applying CLAHE directly to the resized original image. The comparison showed that the difference was not very large, so CLAHE alone was considered enough for the preprocessing stage.

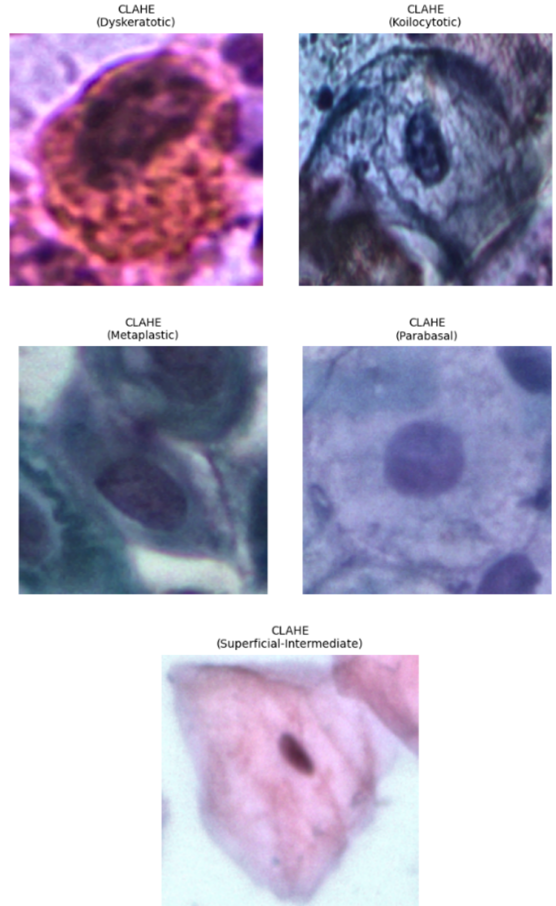

Fig. 4: CLAHE processed images before applying median filtering across the five cell classes

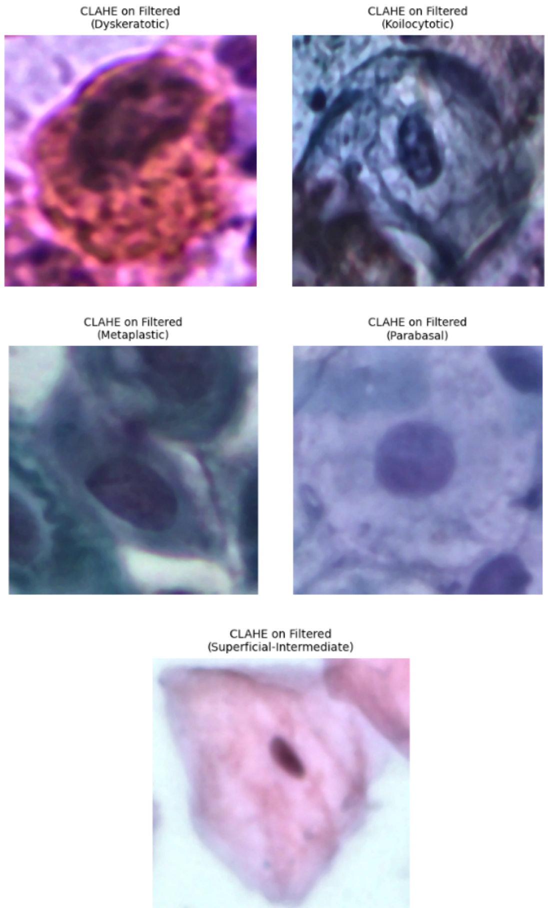

Fig. 5: CLAHE processed images after applying median filtering across the five cell classes

B. Segmentation

In this stage of the project, segmentation was performed to separate the cervical cell region and the nucleus region from the microscopic cell images. Segmentation is an important preprocessing step because cervical cell classification depends not only on the whole-cell appearance, but also on the visual properties of the nucleus. Especially in cytology images, nuclear size, shape, and location provide meaningful information for distinguishing between normal and abnormal cell classes.

In this progress report, different segmentation approaches were tested. First, the available nucleus coordinate annotations in the `nuc.dat` files were used to generate ground truth nucleus masks. Then, Otsu thresholding was tested as a preliminary cell segmentation method. However, because the images have different staining characteristics, uneven illumination, and low contrast between the cell body and the background, Otsu thresholding was not selected as the final segmentation method. Therefore, a non-Otsu segmentation pipeline based on GrabCut and K-Means clustering was used to obtain the cell and nucleus masks.

1) Nucleus Ground Truth Mask Generation from `nuc.dat` *Files:* The SIPaKMeD dataset contains coordinate-based annotation files for the nucleus region. These files are stored as `nuc.dat` files and include the boundary coordinates of the nucleus for each corresponding cell image. In this step, the coordinate points were read from the `nuc.dat` files and converted into binary masks. The polygon formed by the coordinate points was filled to obtain the ground truth nucleus region.

```
points = np.loadtxt(nuc_dat_path)

mask = np.zeros((img_height, img_width), dtype=np.
    uint8)
polygon = points.astype(np.int32)

cv2.fillPoly(mask, [polygon], 255)
```

Listing 3: Generation of ground truth nucleus mask from `nuc.dat` coordinates

Listing 3 shows how the nucleus boundary coordinates from the `nuc.dat` file were converted into a binary ground truth mask. The coordinate points were used to form a polygon, and this polygon was filled to represent the nucleus region.

Figure 6 shows one randomly selected image from each class. For each sample, the original image, the generated binary nucleus mask, the nucleus contour displayed on the original image, and the filled overlay are shown. The blue contour and filled region indicate the nucleus area obtained from the annotation file.

The results show that the `nuc.dat` files provide useful ground truth information for the nucleus. The generated masks generally match the visible nucleus locations in the original images. Therefore, these masks were used as reference masks for evaluating the predicted nucleus segmentation outputs.

2) Preliminary Cell Segmentation Trial using Otsu Thresholding: Before applying the final non-Otsu segmentation pipeline, Otsu thresholding was tested as a preliminary cell segmentation method. Otsu thresholding is an automatic thresholding technique that separates an image into foreground and background regions based on grayscale intensity values. It was tested because it is simple to apply and can work well when the object and background have clear intensity differences.

Figure 7 shows the preliminary cell segmentation results obtained using Otsu thresholding. Each row includes the original image, grayscale image, Otsu mask, and segmented output.

The results show that Otsu thresholding was not sufficiently reliable for this dataset. In some samples, the method selected background regions instead of the actual cell body. In other samples, only a part of the cell region was segmented. This limitation is mainly related to uneven illumination, staining differences, and weak contrast between the cell region and the background. Therefore, Otsu thresholding was not used as the main segmentation method in this progress report.

3) Cell and Nucleus Segmentation without Otsu Thresholding: After observing the limitations of Otsu thresholding, a non-Otsu segmentation approach was applied to extract the cell and nucleus regions automatically. In this approach, GrabCut was used for cell segmentation, while K-Means clustering was used for nucleus segmentation. GrabCut was selected because it can separate the foreground cell region from the background by using color and spatial information. K-Means clustering was applied to the nucleus region because the nucleus generally appears darker than the surrounding cytoplasm.

Figure 8 shows the cell and nucleus segmentation results obtained without using Otsu thresholding. The red contour represents the segmented cell boundary, while the green contour represents the segmented nucleus boundary.

The results show that the non-Otsu approach produced more meaningful cell masks compared with the preliminary Otsu thresholding trial. The cell boundaries were generally close to the visible cell body, especially in the Metaplastic, Parabasal, and Superficial-Intermediate samples. The nucleus segmentation also gave reasonable results in several images. However, some limitations were still observed. In some samples, the nucleus mask was affected by dark background regions, staining differences, or nearby artifacts. This shows that segmentation performance depends strongly on image contrast, staining quality, and background complexity.

4) Nucleus Segmentation Evaluation using Dice Score: To evaluate the nucleus segmentation results, the predicted nucleus masks were compared with the ground truth masks generated from the `nuc.dat` files. The Dice similarity coefficient was used as the evaluation metric. Dice Score measures the overlap between the predicted mask and the ground truth mask. It is calculated as

$$Dice = \frac{2|A \cap B|}{|A| + |B|} \tag{1}$$

where A represents the predicted nucleus mask and B represents the ground truth nucleus mask. A Dice Score close to 1 indicates strong overlap, while a Dice Score close to 0 indicates poor overlap.

```python
intersection = np.logical_and(pred_mask, gt_mask).sum()

dice_score = (2.0 * intersection) / (
    pred_mask.sum() + gt_mask.sum() + 1e-8
)
```

Listing 4: Dice Score calculation for nucleus mask evaluation

Listing 4 shows the Dice Score calculation used to evaluate the overlap between the predicted nucleus mask and the ground truth nucleus mask. A small value was added to the denominator to avoid division by zero.

Figure 9 shows the comparison between the ground truth nucleus masks and the predicted nucleus masks. In the visualization, green regions represent the ground-truth-only regions, red regions represent the predicted-only regions, and yellow regions represent the overlapping area between the ground truth and prediction.

The Dice Score results show that the nucleus segmentation performed well for some classes but failed in certain samples. The Superficial-Intermediate, Parabasal, and metaplastic samples achieved high Dice scores, approximately 0.9334, 0.8983, and 0.8818, respectively. The koilocytotic sample achieved a moderate Dice Score of approximately 0.6900. However, the Dyskeratotic sample yielded a Dice Score of 0.0000 because the predicted nucleus mask failed to correctly detect the nucleus region in that image.

These results show that coordinate-based ground truth masks can be successfully generated from the `nuc.dat` files. However, automatic nucleus segmentation still needs improvement for difficult samples. Overall, the preliminary Otsu thresholding trial revealed the limitations of simple threshold-based segmentation, while the non-Otsu GrabCut and K-Means-based approaches produced more meaningful cell and nucleus segmentation results. For the next stage of the project, the segmentation pipeline can be improved by applying better preprocessing, adaptive color-based filtering, morphological operations, or deep learning-based segmentation methods.

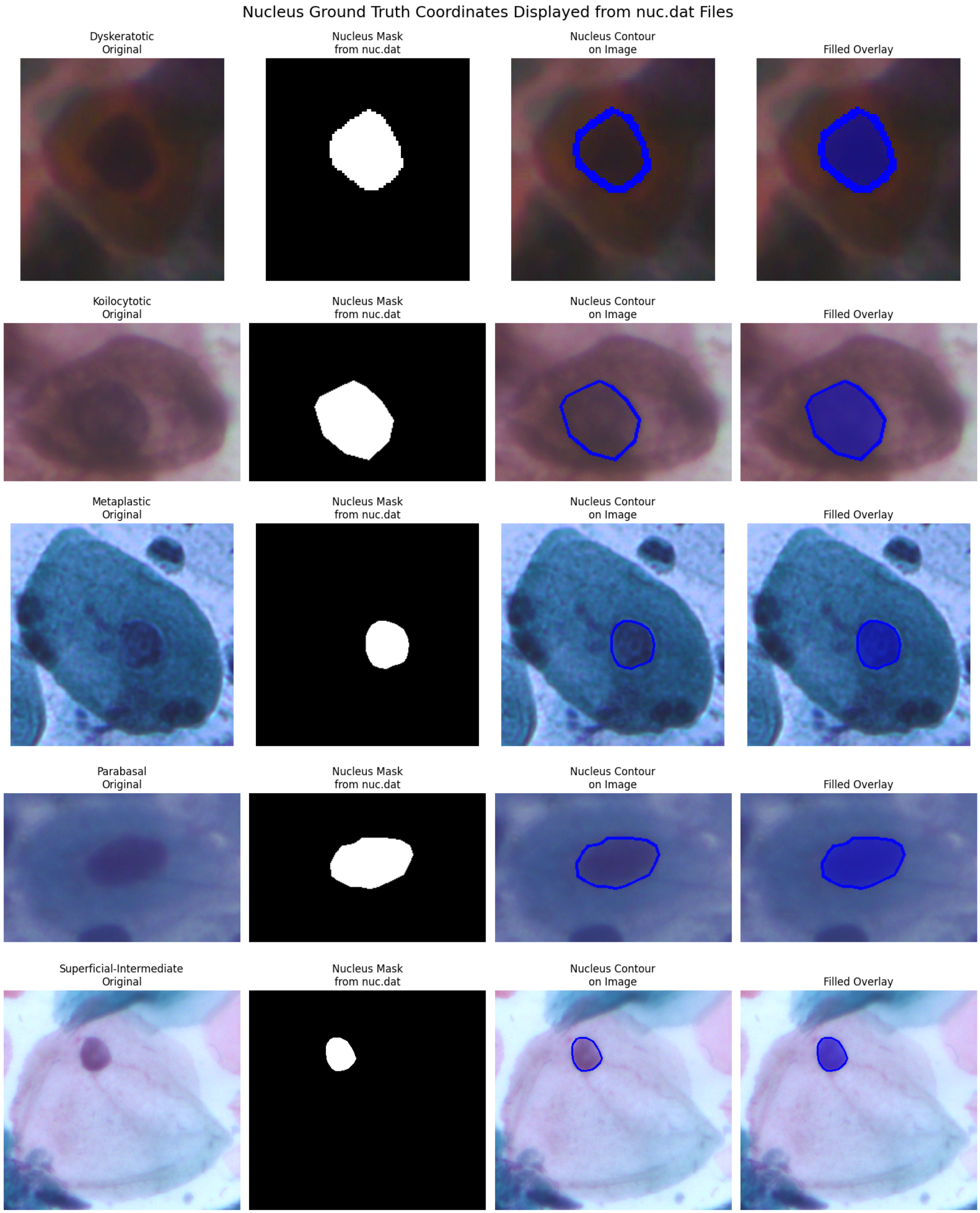

Fig. 6: Nucleus ground truth masks generated from `nuc.dat` coordinate files for five cervical cell classes.

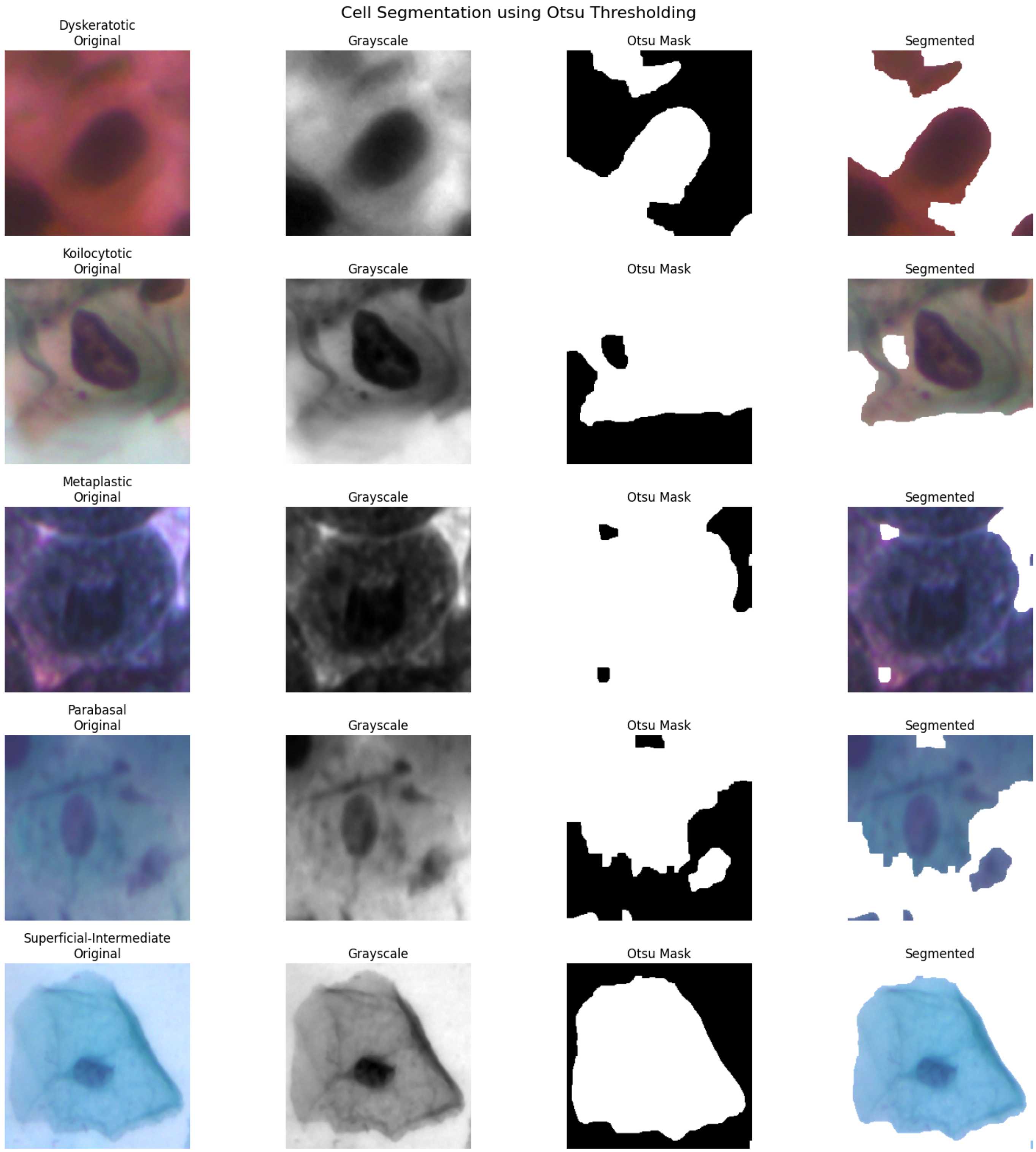

Fig. 7: Preliminary cell segmentation results obtained using Otsu thresholding.

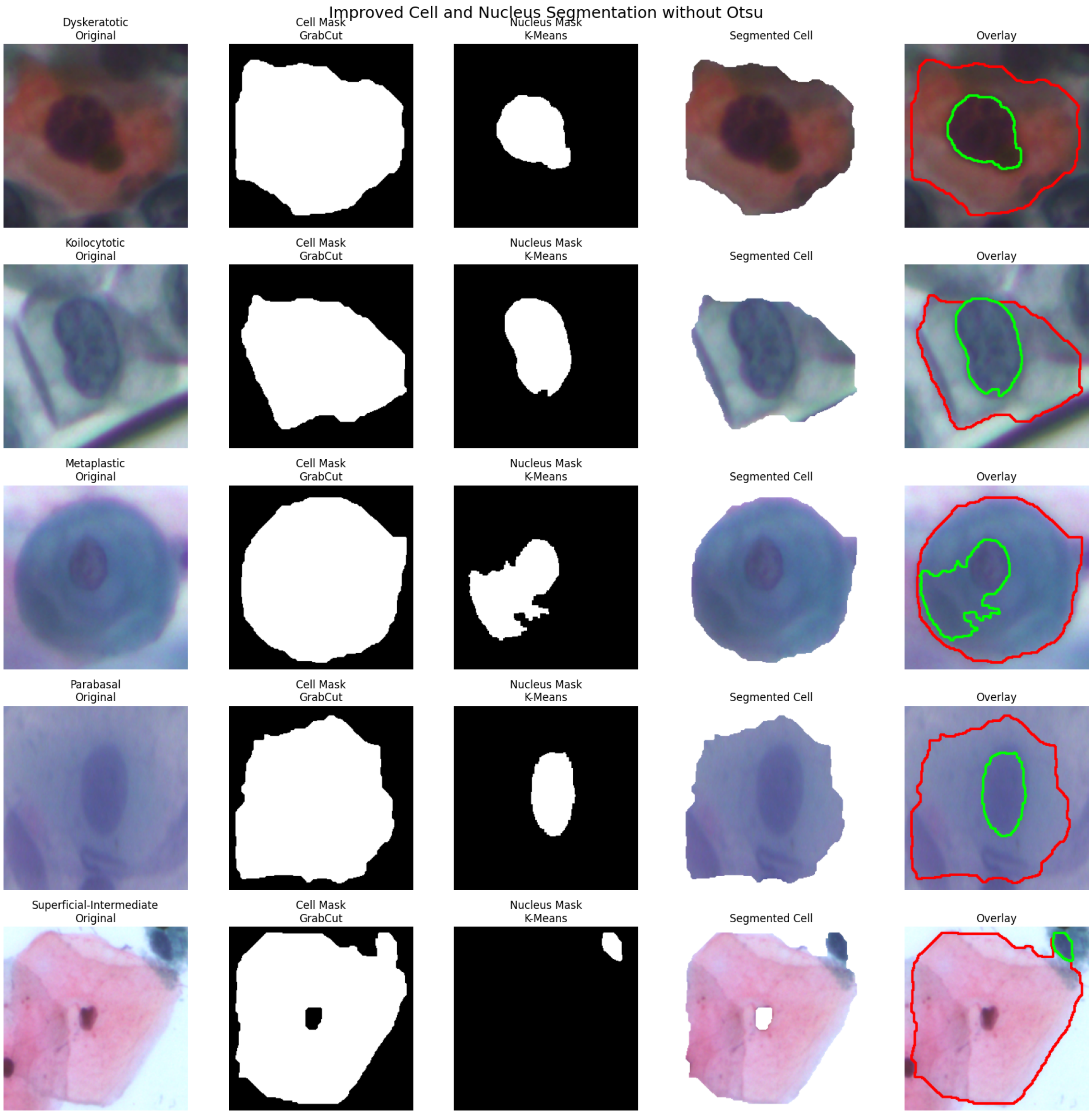

Fig. 8: Cell and nucleus segmentation results obtained without Otsu thresholding. Red contours show cell boundaries, and green contours show nucleus boundaries.

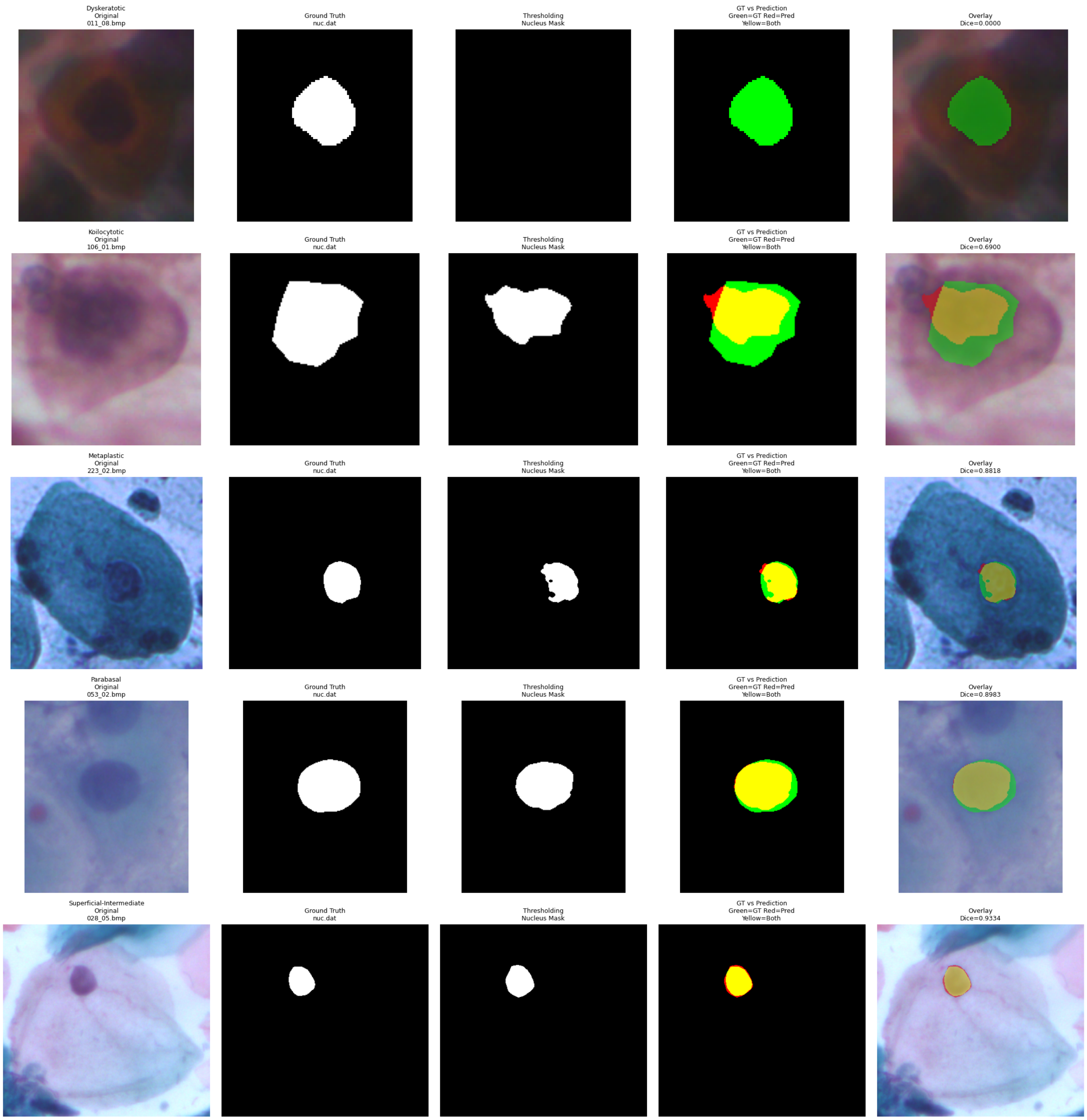

Fig. 9: Comparison of ground truth and predicted nucleus masks using Dice Score. Green indicates ground-truth-only regions, red indicates predicted-only regions, and yellow indicates overlapping regions.